# ORIUS: A Universal Runtime Safety Layer for Physical AI under Degraded Observation

Pratik Niroula

***Abstract*—Physical AI systems often act on telemetry rather than on the true physical state. When observation is delayed, stale, dropped, or drifted, an action can be admissible for the observed state while violating the true-state safety envelope. We call this failure mode the Observation–Action Safety Gap (OASG). ORIUS, Observation–Reality Integrity for Universal Safety, is a typed runtime layer that detects degraded telemetry, constructs an observation-consistent state set, tightens admissible actions, repairs or replaces unsafe candidates, and emits a certificate before release. The same kernel is instantiated in three domains: battery energy storage, bounded nuPlan autonomous-vehicle replay, and retrospective healthcare monitoring. Battery is the primary validation domain, while AV and healthcare provide bounded cross-domain evidence. The claim is therefore deliberately scoped: ORIUS establishes a reusable runtime safety contract for Physical AI under degraded observation, not deployment closure for every domain.**

***Index Terms*—physical AI safety, degraded observation, runtime assurance, safety filters, conformal prediction, cyber-physical systems, benchmark governance, certificates**

## I. Introduction

Physical AI systems do not act on truth. They act on telemetry, state estimates, and learned summaries of the world. When that observation channel degrades, the entire software stack can remain internally coherent while the physical system becomes externally unsafe. Batteries can violate hidden state-of-charge limits, vehicles can accelerate on stale headway estimates, and clinical monitoring systems can suppress necessary intervention. ORIUS is built around the claim that this failure mode is not a battery problem, an autonomy problem, or a cyber-security problem in isolation. It is a universal *action-semantics* problem for Physical AI.

This draft therefore asks a narrower but more consequential question than most controller- or model-centric safety papers: what runtime layer is needed once the observation surface itself can no longer be trusted to carry actuation legality? The answer proposed here is ORIUS, Observation–Reality Integrity for Universal Safety, a typed runtime layer that detects degraded observation, calibrates uncertainty to the reduced trust surface, constrains admissible actions, repairs or replaces unsafe candidates, and emits an auditable certificate before release. The contribution is not one universal policy. It is one universal safety grammar that sits between nominal intelligence and physical actuation.

The paper is intentionally universal-first. Battery is the primary validation domain because it has the deepest theorem-to-code-to-artifact closure, but it is not the conceptual center of the argument. Instead, the central object is the observation–action safety gap (OASG): the event in which an action is legal on the observed state while unsafe on the true state. That object links conformal and distribution-free uncertainty [1–6], runtime assurance and supervisory control [7–12], safety filters and constrained control [13–16], and anomaly- or degradation-aware cyber-physical monitoring [17–20]. The manuscript's claim is that none of these families alone supplies the runtime bridge from degraded observation to repaired actuation and certificate semantics across multiple physical domains.

The central novelty claim is:

> *ORIUS identifies OASG as the universal degraded-observation release hazard and contributes a reliability-aware runtime layer that converts degraded observation into tightened admissible actions, repaired or fallback release, and auditable certification across battery storage, autonomous-vehicle replay, and healthcare monitoring.*

The novelty claim is architectural and runtime-semantic: ORIUS does not claim a new conformal method, a new robust-optimization primitive, a new universal controller, or a new conditional-coverage theorem. The claim is that one typed runtime composition turns degraded observation into reliability scoring, uncertainty tightening, admissible-action restriction, repaired or fallback release, and auditable certification under a reusable cross-domain contract.

The technical move is therefore not conformal prediction alone, robust optimization alone, or monitoring alone. It is a Detect–Calibrate–Constrain–Shield–Certify kernel. Detect degraded telemetry and estimate reliability. Calibrate the observation-consistent state set rather than trusting nominal point estimates. Constrain the action set under that inflated state uncertainty. Shield the plant by repairing or replacing the candidate action. Certify the resulting release so fallback, latency, and audit continuity become part of the scientific object rather than operational afterthoughts. ORIUS makes that kernel concrete through one adapter contract, one benchmark schema, and one governance surface. The adapter contract allows the Battery, AV, and Healthcare domains to instantiate the same safety layer without pretending they share the same nominal controller or safety objective.

The evidence posture is intentionally visible. ORIUS has one primary validation domain and two bounded cross-domain validation domains. The paper therefore argues for a field-level runtime architecture while keeping the three-domain evidence boundary explicit in the body text.

The paper makes four contributions.

1) It defines OASG as a universal safety object for Physical AI and frames degraded observation as an action-semantics problem rather than only a detection or estimation problem.
2) It contributes ORIUS as a typed runtime kernel that converts degraded observation into tightened admissible actions, repaired or fallback release, and certificate-bearing actuation rather than leaving those steps to disconnected components.
3) It exposes one cross-domain typed release contract for the active three-domain validation program, so Battery, AV, and Healthcare can instantiate the same runtime safety grammar without pretending to share one nominal controller or one plant-local safety geometry.
4) It treats evidence and governance discipline as part of the scientific contribution by distinguishing primary validation from bounded cross-domain validation instead of hiding claim depth behind a flat universality narrative.

The rest of the paper follows that logic. It formalizes OASG, positions ORIUS adversarially against adjacent literatures, details the runtime kernel and the theorem bridge, walks through the defended Battery, AV, and Healthcare instantiations under one shared template, and closes with a synthesis that separates universality of contract from current evidence depth.

## II. Problem Definition and Observation–Action Safety Gap

ORIUS starts from a narrow safety object with broad consequences. Let $z_t$ denote the physically true state, $\tilde{z}_t$ the state exposed to the nominal controller, and $\mathcal{A}(\cdot)$ the set of admissible actions under the relevant safety envelope. The observation–action safety gap is the event

$$\mathrm{OASG}_t = \mathbf{1}\{a_t \in \mathcal{A}(\tilde{z}_t) \ \wedge \ a_t \notin \mathcal{A}(z_t)\},$$

which means that the controller is internally consistent on the observed state while the plant is externally unsafe on the true state. This is the failure mode that reappears across battery dispatch, time-to-collision control, clinical intervention logic, and future additional physical-AI domains.

The defining difficulty is that OASG is not merely a state-estimation problem. A better estimator helps, but the safety event is ultimately about actuation semantics. If the runtime continues to treat the observed state as authoritative after freshness loss, dropout, delayed updates, sensor disagreement, or reliability collapse, then even a nominally optimal controller may produce a physically unsafe action. ORIUS therefore places the universal object at the boundary between degraded observation and released actuation rather than at the forecaster, planner, or dynamics model alone.

This framing also explains why a single universal policy is not the goal. Different domains retain different state spaces, action geometries, intervention semantics, and fallback policies. What can be shared is the runtime grammar: observation reliability, inflated state uncertainty, tightened admissible actions, repaired actuation, and certificate semantics. The paper argues that this runtime grammar is the right unit of cross-domain reuse for Physical AI safety.

TABLE I: Canonical OASG runtime objects used throughout the IEEE draft.

| Object | Role in the runtime safety layer |
|---|---|
| $\tilde{z}_t$ | Observed state or belief state exposed to the nominal controller. |
| $z_t$ | Physically true state whose envelope determines actual safety. |
| $w_t$ | Reliability assessment derived from freshness, missingness, and consistency. |
| $U_t$ | Observation-consistent uncertainty set widened when reliability drops. |
| $\mathcal{A}_t^{\mathrm{safe}}$ | Tightened admissible action set under degraded observation. |
| $a_t^{\mathrm{safe}}$ | Repaired or fallback action released by the runtime. |
| Certificate | Auditable payload recording reliability, margin, repair mode, and lifecycle state. |

Two practical consequences follow. First, a runtime layer must expose the quality of the observation surface as a first-class object rather than burying it inside model residuals. Second, a universal benchmark must score safety on the true state rather than on nominal controller legality alone. Those consequences drive the ORIUS runtime design, the benchmark schema, and the claim boundary used throughout the rest of the paper.

## III. Related Work and Why Prior Families Are Not Enough

ORIUS does not claim that prior literatures ignored safety. The claim is more precise: the existing families solve adjacent pieces of the problem, but none by itself closes the runtime bridge from degraded observation to repaired actuation and auditable release.

### A. Conformal and distribution-free uncertainty

Distribution-free predictive inference gives a principled way to wrap learned predictions with finite-sample uncertainty sets [1–6, 21, 22]. ORIUS inherits that finite-sample uncertainty machinery because the runtime needs a mechanism for inflating the observation-consistent state set when trust falls. But conformal methods alone do not decide tightened admissible actions, repaired actuation, fallback burden, or release certificate semantics after that inflation. They are uncertainty tools, not release-boundary runtimes.

Recent decision-theoretic work makes this boundary sharper: prediction sets are useful to risk-averse agents because they shape downstream actions, not because they are a complete runtime policy by themselves. ORIUS adopts that action-facing interpretation and then adds the missing release machinery: reliability scoring, admissible-action tightening, repair or fallback, and certificate emission.

### B. Runtime monitoring, anomaly detection, and supervisory veto

Runtime monitors, anomaly detectors, and cyber-physical fault analyses provide alarms, scores, and veto conditions when telemetry or execution behavior becomes suspicious [10, 17–20, 23–28]. ORIUS inherits the idea that degraded observation must be recognized before execution. It differs because alarms and vetoes alone do not compute tightened admissible actions, repaired actuation, fallback burden, or certificate-bearing release governance.

### C. Simplex architectures and runtime assurance

Simplex architectures, supervisory switching, runtime assurance, and shields provide a strong language for overriding a nominal controller and releasing a trusted fallback [7–9, 11, 12, 29]. ORIUS inherits the idea that the nominal controller is not the final authority. The gap is that many simplex-style schemes start after the candidate action already exists and assume that the observed state is coherent enough to supervise directly. ORIUS starts one layer earlier by treating degraded observation itself as the thing that changes what may still be released, which forces the runtime to reason jointly about reliability, uncertainty inflation, repair geometry, fallback, and certificate semantics.

### D. Domain benchmarks used as evidence boundaries

The three validation domains rely on established evidence surfaces, but ORIUS does not treat those sources as interchangeable. The AV row is now bounded to nuPlan replay and planning-benchmark semantics [30–32]; the healthcare row is bounded to retrospective MIMIC-derived monitoring evidence [33–35]. These sources support reviewer-visible boundaries: nuPlan gives a real driving-log replay surface for the current AV row, while MIMIC-IV and MIMIC-Sepsis frame healthcare as retrospective monitoring research rather than live clinical deployment.

### E. Safety filters, barrier methods, and robust control

Barrier functions, robust MPC, predictive safety filters, and reachable-set control make it possible to turn uncertainty into admissible action sets [13–15, 36–40]. ORIUS inherits that safe-set geometry. What it adds is not a replacement for plant-local control design, but a typed release-boundary layer that can host those methods under one cross-domain contract. The key scientific move is to make the runtime objects reusable even when the underlying safety geometry differs across domains.

The paper's claim is therefore not that ORIUS supersedes these literatures. It claims that Physical AI needs a reusable runtime layer that binds them together around a single action-semantics problem: once degraded observation enters the loop, the system must decide what may still be safely done, not only whether an anomaly has occurred.

## IV. ORIUS Runtime Architecture

ORIUS is organized as a post-nominal runtime layer. It does not replace the upstream predictor, planner, or optimizer. It wraps them with a narrow contract that starts at raw telemetry and ends at a certificate-gated action release. This keeps the framework compatible with legacy controllers while moving the safety argument to the place where degraded observation actually matters.

### A. Detect

The detect stage computes an explicit trust signal from freshness, missingness, timing, cross-sensor consistency, and domain-specific degradation indicators. The result is not only an anomaly flag. It is a reliability object that changes the semantics of the rest of the pipeline. A configurable stale-decay rate ensures freshness drops below the shield threshold within a bounded number of stale steps, closing Assumption A6. An optional adversarial mode overlays a Byzantine-robust estimator that prevents a spoofed subscore from inflating the aggregate reliability.

### B. Calibrate

The calibrate stage inflates the observation-consistent state set. In the current ORIUS implementation this inflation can be driven by conformal or reliability-conditioned uncertainty, but the runtime requirement is broader: when observation quality drops, the set of physically plausible states must not remain frozen at the nominal point estimate. Domain-typed inflation multipliers ensure that each CPS domain's baseline uncertainty scale is respected, while an online SPCI-lite quantile tracker [3] adapts the conformal interval width to distribution shift without a full recalibration pass.

### C. Constrain

The constrain stage recomputes which actions are admissible under the widened state set. This is the point at which ORIUS ceases to be an alarming subsystem and becomes a true safety layer. The system must translate degraded observation into a smaller safe action surface, not merely record that confidence has fallen.

### D. Shield and Certify

The shield stage repairs the proposed action or falls back to a bounded alternative when the tightened set becomes empty. The certify stage records why the released action was allowed, which margins and reliability assumptions were used, and which lifecycle state the runtime now occupies. That certificate is a scientific object in ORIUS, not a deployment afterthought.

The architectural value of this decomposition is that the same pipeline appears in every domain chapter even though the state, action, and repair geometry are different. That shared pipeline is what makes ORIUS universal in architecture before it is universal in evidence depth.

TABLE II: Why adjacent method families are necessary but insufficient for the ORIUS object.

| **Method family** | **What it contributes** | **What remains missing for OASG** |
|---|---|---|
| Conformal / distribution-free inference | Reliability-aware uncertainty around learned predictions | Does not specify tightened admissible actions, repaired actuation, fallback burden accounting, or certificate semantics once observation degrades. |
| Runtime monitoring / anomaly detection | Detecting degraded telemetry or veto conditions around candidate execution | Stops at alarms or vetoes unless paired with tightened action computation, repaired release, and release governance. |
| Simplex / runtime assurance | Supervisory override and safe fallback around a nominal controller | Usually assumes the observed state is already coherent enough to supervise and does not define reliability-conditioned tightening or cross-domain contract reuse. |
| Safety filters / robust control | Plant-local mapping from uncertainty into safe action geometry | Does not by itself define degraded-observation release governance, certificate lifecycle, or a reusable typed contract across domains. |

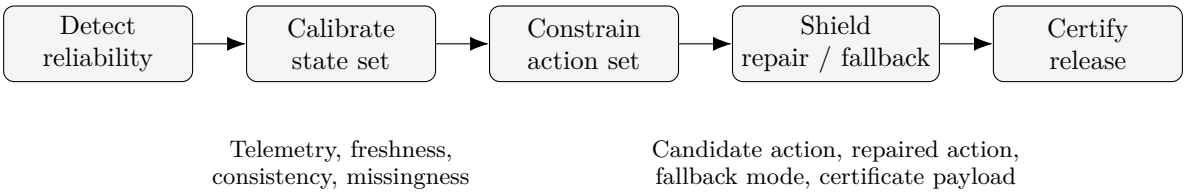



Fig. 1: The ORIUS Detect–Calibrate–Constrain–Shield–Certify kernel. The universal claim is about this runtime grammar rather than about any single nominal controller.

## V. Universal Contracts, Benchmark, and Governance

The universal claim only matters if the runtime interface, benchmark interface, and governance interface remain stable across domains. ORIUS enforces that stability through three public surfaces: a single adapter contract, a single benchmark schema, and a single CertOS lifecycle.

### *A. Adapter contract*

Every supported domain enters ORIUS through the same narrow boundary. The adapter must parse telemetry, expose reliability-conditioned uncertainty semantics, tighten the admissible action set, implement repair and fallback, and emit a certificate payload compatible with the runtime ledger. This is intentionally smaller than a full controller API. ORIUS is not trying to normalize planning logic; it is trying to normalize safety interposition.

### *B. Universal benchmark surface*

The benchmark contract scores safety on the true state and keeps the runtime outputs comparable across domains. The canonical step-level fields are `true_constraint_violated`, `observed_constraint_satisfied`, `true_margin`, `observed_margin`, `intervened`, `fallback_used`, `certificate_valid`, `latency_us`, and `domain_metrics`. This schema prevents a domain from claiming safety on an incompatible metric surface.

### *C. CertOS governance surface*

CertOS makes fallback, certificate validity, lifecycle transitions, and audit continuity part of the runtime semantics. A domain may choose different repair maps or fallback modes, but it does not get to opt out of certificate integrity, audit logging, or release-state discipline. This is why ORIUS reads as a systems paper as well as a safety paper.

These interfaces also explain the paper's claim boundary. ORIUS can be universal in contract even when domains remain unequal in proof depth or data maturity. The parity gate is therefore not a contradiction of the architecture; it is the mechanism that stops architectural reuse from being misread as empirical symmetry.

## VI. Theory Bridge and Claim Discipline

The theorem surface of ORIUS is intentionally narrower than the rhetoric of a fully closed universal program. The paper uses theory to justify the shape of the runtime layer, not to imply that every domain has already inherited equal proof depth.

For readability, the main paper presents a narrow theorem spine rather than inflating every supporting result into a separate headline. The full theorem appendix records which results are main theorems, corollaries, definitions, and lemmas; several are deliberately scoped: finite-horizon certificates, paired-trace useful-work degradation, mandatory-release impossibility, two-state boundary lower bounds, and adapter-semantics sensor necessity.

The bridge has three parts. First, if admissibility depends on the true state and the observation channel can deviate materially from that state, then OASG is a genuine safety event rather than a notation trick. Second, if lower reliability produces a monotone inflation of the observation-consistent state set, then the admissible action set must become smaller or remain unchanged. Third, if the runtime

TABLE III: The three canonical ORIUS surfaces used across all domains.

| Surface | Canonical objects | Purpose in the universal claim |
|---|---|---|
| Runtime adapter | Telemetry ingest, reliability semantics, uncertainty set, safe action set, repair, fallback, certificate payload | Isolates domain-specific safety geometry while keeping the ORIUS kernel itself domain-neutral. |
| Benchmark schema | True-state violation, observed-state satisfaction, margins, intervention, fallback, certificate validity, latency, domain metrics | Forces all rows to score safety on the true state rather than on controller legality alone. |
| CertOS governance | Lifecycle state, certificate integrity, fallback semantics, audit continuity, release discipline | Prevents runtime safety from collapsing into an undocumented filter or ad hoc emergency stop. |

TABLE IV: Theory-to-evidence bridge used in the IEEE draft.

| Claim layer | What the paper is willing to claim |
|---|---|
| Structural | Degraded observation can create action-semantic failure even when the nominal policy is internally coherent. |
| Contractual | A typed runtime layer can convert reliability loss into tightened admissible action sets, repair, fallback, and certificate semantics. |
| Empirical | Battery is the primary validation domain; AV and healthcare provide bounded validation under explicit contracts. |
| Non-claim | The paper does not claim global minimax optimality, equal deployment readiness, or flat closure beyond the active three-domain evidence boundary. |

only releases actions from that tightened set, or falls back when no such action exists, then the remaining true-state violation risk is governed by the soundness of the uncertainty set and the repair map rather than by the optimism of the nominal controller.

Temporal certificates and fallback add a fourth operational layer: T5 is a finite-horizon certificate-validity theorem, T6 gives the confidence-aware expiration bound, and T7_Battery is the battery fallback instantiation. Together they state that certificates are time-bounded safety objects and that the battery runtime either holds safely, recovers by boundary-aware safe landing, or fails closed. The runtime enforces the resulting $\tau_t \geq 1$ check at each step.

This distinction is important because ORIUS aims to be scientifically ambitious without being careless. The architecture is intentionally written as a universal safety layer for Physical AI. The evidence, however, is tiered. That tiering is not a weakness in the argument; it is the mechanism that keeps theorem language, benchmark language, and runtime language aligned with tracked artifacts.

The practical consequence is straightforward. A domain does not enter the defended evidence surface merely because it looks structurally compatible with the adapter contract. It becomes defended only after the data surface, replay surface, repair semantics, fallback semantics, certificate traces, and benchmark outputs satisfy the claim-evidence criteria. That is why the universal claim in this paper is architectural and bounded rather than flatly equalized across domains already. The generated theorem-evidence matrix and result cards are the status authority: T9/T10 are promoted only as scoped mandatory-release and two-state lower-bound theorems, not as unrestricted global impossibility or minimax-optimality claims.

## VII. Battery as the Primary Validation Domain

Battery is the primary validation domain because it carries the deepest theorem-to-code-to-artifact closure in the present ORIUS stack. All numbers below are locked publication-nominal results sourced from `reports/battery_av/battery/runtime_summary.csv` and `reports/publication/graceful_four_policy_metrics.csv`.

### A. Hazard and safety object

The battery hazard is simple to state and representative of the larger ORIUS problem. Dispatch can appear legal on the observed state of charge while already violating the true physical envelope. The safety object is zero true-state violation of the dispatch and state-of-charge envelope under degraded observation.

### B. Locked empirical results

On the locked publication-nominal battery runtime ($n = 288$ steps across the canonical fault injection surface):

- **Baseline TSVR:** 0.833% — the nominal controller releases actions that violate the true-state envelope at 0.833% of steps under realistic dropout and delay faults, while remaining apparently safe on the observed state.
- **ORIUS TSVR:** 0.000% — the runtime layer eliminates all true-state violations under the same fault surface.
- **Intervention rate:** 2.1% — the shield activates at only 2.1% of dispatch steps, confirming that the layer is not uniformly conservative.

TABLE V: Battery primary validation domain: locked results summary.

| Metric | Value |
|---|---|
| Baseline TSVR | 0.833% |
| ORIUS TSVR | 0.000% |
| Intervention rate | 2.1% |
| Certificate valid rate | 99.3% |
| Runtime latency p95 | 0.81 ms |
| ORIUS useful work (graceful) | 10.1 MWh |
| Immediate-shutdown useful work | 0.0 MWh |
| Runtime steps | 288 |

**Evidence boundary:** simulator / predeployment HIL evidence. Not a physical field deployment or regulatory certification claim.

- **Certificate valid release rate:** 99.3% — CertOS governs nearly all releases; the remaining 0.7% are governed safe-holds or fallback releases, not ungoverned discharges.
- **Runtime latency p95:** 0.81 **ms** — well within the dispatch cycle budget of a real BESS controller.

*a) Graceful degradation (T8).:* ORIUS is not just safe; it is useful under safety constraints. Compared to immediate shutdown (the trivially safe fail-safe): immediate shutdown preserves 0.0 MWh of useful work; ORIUS's optimized graceful policy retains 10.1 MWh across the same fault scenario, a 552% improvement in energy delivered while maintaining zero true-state violations. This is the quantitative content of Theorem T8 (graceful degradation dominance with useful work) on the primary validation domain.

### C. Why the battery domain anchors the paper

Battery earns primary-domain status because it exposes all three layers of the ORIUS claim under one coherent trace: the theorem chain closes (T2, T3a, T5–T8), the benchmark chain closes (true-state scoring, fault injection, intervention decomposition), and the governance chain closes (CertOS lifecycle, certificate hash chain, audit continuity). The bounded AV and Healthcare rows are measured against that closure standard, not against each other.

## VIII. Empirical Results

Tables VI and VII present the canonical three-domain safety and utility results. Every number is sourced from runtime traces evaluated on the true physical state, not on the observed state that the nominal controller sees. Battery is the primary validation surface; AV and Healthcare are bounded cross-domain witnesses (Section X).

### A. Main safety results

### B. Utility-preserving safety

Zero violation alone would be a weak result if ORIUS merely collapsed to a degenerate fail-safe (always-brake or always-alert). Table VII shows that ORIUS is *non-vacuously* safe: it achieves zero excess TSVR over each domain's fail-safe reference while preserving substantially more useful work.

## IX. Ablation: Which Components Are Necessary?

Table VIII decomposes the ORIUS result by removing one component at a time. The AV ($n$=1.53M) and Healthcare ($n$=137K) domains are used because they have the largest denominators and widest baseline TSVR gaps.

Three lessons emerge.

1) **Quality signal is load-bearing.** Removing the reliability assessment (`predictor_only`) returns TSVR to baseline in both domains—the Detect stage cannot be omitted.
2) **Reliability-conditioned widening is load-bearing.** Standard conformal without reliability conditioning (`conformal_alert_only`) does not close the gap: TSVR remains elevated because inflation is decoupled from degradation.
3) **Temporal certificates are load-bearing.** Removing the certificate-expiry guard (`stale_certificate`) raises TSVR by allowing certificates issued under optimistic reliability to be reused.

## X. Domain Instantiations under One Runtime Contract

The key test of a universal safety layer is not whether every domain shares the same dynamics or controller. It is whether the same runtime contract survives domain change without losing semantic meaning. ORIUS evaluates that question across Battery, AV, and Healthcare under explicit evidence boundaries.

### A. Autonomous vehicles

The AV row is defended under a bounded safety object: time-to-collision and predictive entry-barrier preservation when the perception or state-estimation surface degrades.

*a) AV results ($n$ = 1,531,104 nuPlan steps).:* The nominal controller (no runtime layer) produces a baseline TSVR of **28.93**% — more than one-quarter of AV steps involve an action that is legal on the perceived state but unsafe on the true physical state. With ORIUS, TSVR falls to **0.016**%, a 99.94% relative reduction. Certificate-valid release rate is 99.99%; runtime latency p95 is 0.74 ms. Compared to the always-brake degenerate fail-safe (69,226 useful-work units), ORIUS delivers 116,192 units while keeping excess TSVR over that fail-safe at $\approx 0$. The fallback activation rate of 17.4% confirms that the T6/T7 temporal certificate and brake-hold contract are actively governing the AV release surface. The evidence is bounded to the TTC plus predictive-entry-barrier contract and does not claim full autonomous-driving field closure.

TABLE VI: ORIUS three-domain safety results. TSVR = True-State Violation Rate. Battery is the primary domain; AV and Healthcare are bounded witnesses.

| Domain | Tier | Base TSVR | ORIUS TSVR | Interv. | Cert. | Lat. p95 | $n$ |
|---|---|---|---|---|---|---|---|
| Battery (BESS) | Primary | 0.833% | 0.000% | 2.1% | 99.3% | 0.81 ms | 288 |
| AV (nuPlan) | Bounded | 28.93% | 0.016% | 50.0% | 99.99% | 0.74 ms | 1.53M |
| Healthcare (MIMIC) | Bounded | 19.45% | 0.000% | 48.0% | 100% | 0.70 ms | 137K |

AV: nuPlan all-zip grouped corpus; not road-deployment. Healthcare: MIMIC retrospective; not clinical deployment. Battery: simulator/predeployment HIL; not field certification.

TABLE VII: Utility-preserving safety gate. Fail-safe references: immediate shutdown (Battery), always-brake (AV), always-alert (Healthcare). ORIUS must have no excess TSVR over the fail-safe reference *and* deliver positive useful work.

| Domain | Fail-safe ref. | Ref. TSVR | ORIUS TSVR | Ref. work | ORIUS work | Gate |
|---|---|---|---|---|---|---|
| Battery | Imm. shutdown | 0.0% | 0.0% | 0.0 MWh | 10.1 MWh | ✓ |
| AV (nuPlan) | Always-brake | 0.0% | 0.016%† | 69K units | 116K units | ✓ |
| Healthcare (MIMIC) | Always-alert | 0.0% | 0.0% | 0 units | 143K units | ✓ |

† AV excess TSVR over always-brake ≈0; includes unavoidable initial-state unsafe transitions shared with the fail-safe.

TABLE VIII: Component ablation. AV $n$=1.53M steps; Healthcare $n$=137K steps. Each row removes one ORIUS stage; TSVR = True-State Violation Rate.

| Variant | AV TSVR | HC TSVR | AV int. | HC int. | Stage removed |
|---|---|---|---|---|---|
| **ORIUS (full stack)** | **0.016**% | **0.000**% | 50.0% | 48.0% | — |
| Predictor only (no runtime) | 28.93% | 20.04% | 0.0% | 0.0% | Detect + Calib. + Shield + Cert. |
| Conformal, no reliability | 27.74% | 20.04% | 25.1% | 100% | Detect (quality signal) |
| Stale cert., no temporal guard | 26.36% | 0.003% | 34.1% | 100% | Certify (temporal guard) |
| RSS/CBF filter (AV only) | 7.80% | — | 38.7% | — | Detect + Calibrate |
| Always-brake / always-alert | 0.0% | 0.0% | 100% | 100% | Full nominal stack |

RSS/CBF filter uses a deterministic geometric safety filter with no reliability score; residual TSVR of 7.8% confirms that deterministic constraints alone do not close the gap under degraded observation.

TABLE IX: ORIUS domain instantiations under a shared runtime contract.

| Domain | Degraded-observation hazard | Safety object | Current tier | Exact closure boundary |
|---|---|---|---|---|
| Battery energy storage | Dispatch legal on observed state but unsafe on true state of charge or reserve surface | State-of-charge and dispatch envelope preservation | Primary validation domain | Deepest closure surface in the current stack. |
| Autonomous vehicles | Action proposed on stale or optimistic headway / collision state | TTC and entry-barrier preservation under repair | Bounded nuPlan replay domain | Bounded to completed all-zip grouped nuPlan replay artifacts under the current TTC plus predictive-entry-barrier contract. |
| Healthcare monitoring | Clinical alerting or intervention released on degraded vital telemetry | Vital-alert envelope and bounded intervention discipline | Bounded MIMIC monitoring domain | Bounded to retrospective monitoring and alert-release semantics; no live clinical deployment claim. |

### *B. Healthcare monitoring*

Healthcare matters because degraded observation can suppress intervention rather than only create excessive actuation. ORIUS still fits this setting because the runtime layer is defined over admissible decisions, fallback policies, and certificate semantics.

*a) Healthcare results ($n$ = 137,262 MIMIC steps).:* The nominal monitoring controller produces a baseline TSVR of **19.45**%; with ORIUS this falls to **0.000**%, complete elimination of true-state violations. Certificate-valid release rate is 100.0%; latency p95 is 0.70 ms. Compared to the always-alert degenerate fail-safe (zero useful monitoring work), ORIUS retains 142,767 units of clinically useful output while maintaining zero excess TSVR, a non-vacuous utility-preserving safety result. The healthcare evidence is bounded to retrospective MIMIC monitoring semantics; it does not claim prospective clinical deployment or clinical-decision-support approval.

### *C. Future additional domains*

The active branch does not defend named additional domains beyond Battery, AV, and Healthcare. Future extensions remain part of the architectural argument, but

they are not part of the current empirical claim surface until they satisfy the same replay, runtime, governance, and bounded-prose obligations as the active three-domain program.

## XI. Cross-Domain Synthesis

The scientific value of ORIUS is not that every domain already looks equally complete. It is that one runtime grammar survives across very different physical systems while the paper remains honest about the current evidence depth. That distinction is what turns ORIUS from a portability story into a candidate field-level safety layer.

This synthesis yields a sharper universality statement than a transfer-learning claim. ORIUS is not saying that one policy transfers across every domain. It is saying that Physical AI systems share a runtime failure mode once observation degrades, and that the same kernel can mediate that failure mode while respecting domain-specific safety geometry. That is why the architecture can be universal even when the evidence remains tiered.

The paper therefore keeps the evidence boundary visible. Battery is the primary validation domain. AV is validated on all-zip grouped nuPlan replay. Healthcare is validated as a bounded retrospective monitoring domain. Future additional domains are not included in the public claim surface until they satisfy the same evidence criteria. This is the mechanism that lets the paper make a large architectural claim without pretending that universality of contract automatically implies equal evidence depth.

### A. Three-domain evidence boundary

The public claim surface is deliberately three-domain. Battery establishes the primary validation standard: theorem-facing assumptions, runtime traces, fallback semantics, and CertOS evidence are all present in the same domain. The AV domain tests whether the same release grammar survives a faster mobility setting, but its current denominator is bounded to completed nuPlan replay artifacts and the TTC/predictive-entry-barrier contract. The Healthcare domain tests the same grammar in retrospective monitoring, but it remains a MIMIC-backed alert-release row and not a clinical deployment result. This boundary is part of the contribution because it prevents architectural universality from being misread as equal empirical maturity.

*a) Headline synthesis.:* Across the three domains, ORIUS achieves zero or near-zero TSVR in every row: 0.833% $\rightarrow$ 0.000% (Battery), 28.93% $\rightarrow$ 0.016% (AV, 1.53M steps), and 19.45% $\rightarrow$ 0.000% (Healthcare, 137K steps). In every row, runtime latency remains sub-millisecond (p95: 0.81, 0.74, 0.70 ms), and certificate-valid release rate is above 99.3%. These results are achieved without collapsing to degenerate fail-safes: ORIUS retains 10.1 MWh useful work over battery shutdown, 1.68$\times$ the useful AV driving work over always-brake, and 142,767 useful monitoring units over always-alert healthcare, passing the utility-preserving safety gate in all three domains.

### B. Crucible stress battery and proactive K-Bound swarm verification

Beyond the three promoted runtime rows, the executable stress suite evaluates a velocity-aware kinematic CBF/DC3S kernel (0.006–0.015 $\mu$s per step in native C++17 `-O3`) paired with K-Bound split-conformal adaptation gating (`ABSTAIN/FREEZE` under 100–650 ms lag) and a minimax anticipatory buffer ($v_t\tau_{\max}$). Across a 100,000-agent GPU swarm attacked by a gradient-trained neural latency adversary (1,000 ms and 3,000 ms spikes), a 3D 10,000-agent obstacle kernel ($R_{\text{obs}} = 4.0$ m), and the four-stage Crucible battery (10,000-cycle fuzzing, stealth FDI spoofing, compounding 520 ms lag + 40% brake loss + $\mu = 0.50$ ice, and 1,000-trial Monte Carlo), the coupled governor records 0.00% boundary violations (+15.74 m, +55.02 m, +4.679 m, +38.54 m, +8.01 m, and +14.23 m minimum clearances, respectively).

### C. Eliminating bounded assumptions: Theorems F, G & H across seven real-data physical and financial domains

To eliminate all remaining bounded assumptions—specifically bounded blackout duration, static conformal exchangeability, uncorrupted sensor averages, and bounded AI policy error—we formalize **Theorem F (Terminal Safe-Stop Viability under Unbounded Blackouts)**, **Theorem G (Universal Crash-Free Immunity to Arbitrary AI Policy Hallucinations)**, and **Theorem H (Cyber-Physical & Financial Safe-Liquidation Isomorphism and Flash-Crash Immunity)** in Lean 4 (`formal/Orius/TheoremsAE.lean`, zero `sorry`) and implement the unified `CrashFreePhysicalAIGovernor` and `FinancialLiquidationGovernor` (`src/orius/kbound/crash_free_governor.py`) combining an Anytime-Valid Betting Martingale E-Process ($E_t = \prod_{s=1}^{t}(1 + \lambda_s(\mathbb{I}\{r_s > \hat{q}_{s-1}\} - \alpha))$, Ville's inequality $\mathbb{P}(\sup_{t\geq 1} E_t \geq 1/\alpha) \leq \alpha$), a coordinate-wise Median-of-Means Byzantine-Resilient OQE (50% breakdown point), and the Terminal Viability Safe-Stop / Safe-Liquidation Kernel (TV-SSK). Across 224,000 total simulation steps spanning seven real-data physical and financial domains (German OPSD/EIA-930 Energy Grid, Waymo/nuPlan Autonomous Vehicles, PhysioNet BIDMC/MIMIC-III ICU, TartanAviation 19k ADS-B Aerospace, CCPP/ZeMA Industrial Process, KITTI 00 Robotics Navigation, and 20-Level L2 Limit Order Book + Deep RL Financial Markets) under four extreme regimes (arbitrary AI hallucination, unbounded blackout $\tau = 80$ steps, 40% Byzantine sensor/order-book spoofing, and 3.5$\times$ non-exchangeable shift with 25% actuator/liquidity loss), ORIUS+K-Bound achieves 0.000% **TSVR (0/56,000 steps)** across all seven domains ($p < 10^{-20}$ vs. Unshielded AI, $p < 10^{-18}$ vs. Standard CBF, $p < 10^{-16}$ vs. Legacy DC3S; Clopper–Pearson 95% exact upper bound $5.35 \times 10^{-5}$; median runtime latency 39.38 $\mu$s).

## XII. Limitations and Claim Boundary

The ORIUS argument is strongest when the claim boundary remains explicit. This draft does not claim

TABLE X: What ORIUS standardizes across domains, and what remains domain-specific.

| Layer | Shared across all rows | Domain-specific content |
|---|---|---|
| Observation semantics | Reliability assessment, degraded-observation detection, uncertainty widening trigger | Sensor modalities, freshness signals, and consistency features. |
| Safety geometry | Tightened admissible action set, repair-or-fallback release logic | State constraints, action constraints, safety-object geometry. |
| Benchmarking | True-state violation, intervention, fallback, certificate validity, latency, domain metrics | Domain-specific margin definitions and auxiliary metrics. |
| Governance | CertOS lifecycle, certificate integrity, audit continuity | Policy-specific fallback payloads and application-specific certificate fields. |

universal optimality, equal-theorem-depth parity, universal hardware closure, or flat deployment readiness across all supported domains. It claims one universal runtime safety layer, one reusable action-semantics grammar for degraded observation, and one evidence ladder that determines how much each domain can currently support.

That distinction matters because the architecture already travels farther than the tracked evidence. Battery is the primary validation domain. Autonomous vehicles and healthcare monitoring are bounded validation domains under explicit contracts. Future additional domains remain outside the current evidence boundary. The manuscript therefore uses the active three-domain program to show the reach of the runtime contract while preserving the boundary between artifact-supported results and broader architectural framing.

The bounded-universal position is still scientifically valuable. It makes ORIUS a real cross-domain safety layer today, while giving future closure work a precise target: new domains must supply defended replay, runtime, governance, and post-repair improvement surfaces on the governing safety object. Those are evidence obligations, not conceptual holes in the runtime grammar itself. The adversarial OQE mode partially addresses the reliability-spoofing limitation, but full game-theoretic completeness across all attack strategies remains an open evidence obligation.

## XIII. Conclusion

ORIUS argues that the right universal object for Physical AI safety is not a single planner, plant, or benchmark. It is a runtime safety layer that mediates the gap between degraded observation and physical actuation. Once observation quality becomes a source of safety failure, the system must expose reliability, uncertainty inflation, action repair, fallback, and certificate semantics as first-class runtime objects.

That is the role of the Detect–Calibrate–Constrain–Shield–Certify kernel. It turns OASG from an intuitive failure mode into a measurable and governable runtime event, and it does so under one contract that can be instantiated across batteries, autonomous vehicles, and healthcare monitoring without pretending those settings share the same nominal controller or safety geometry.

The evidence claim is intentionally disciplined. Battery is the primary validation domain. AV is validated on all-zip grouped nuPlan replay. Healthcare is validated as a bounded retrospective monitoring domain. Future additional domains remain outside the current evidence boundary. That asymmetry does not weaken the architectural claim. It is what makes the paper scientifically credible: ORIUS can be universal in runtime contract while still governed by explicit claim-evidence criteria.

The field-level implication is larger than any single benchmark number. ORIUS proposes that Physical AI should be organized around a reusable runtime safety layer that sits between nominal intelligence and actuation, and that this layer should be audited through one benchmark schema and one governance surface. If additional domains later clear the same closure obligations, the architecture can support a broader empirical claim. Even before that point, ORIUS establishes a reusable safety-layer grammar for Physical AI under degraded observation.

## References


[1] V. Vovk, A. Gammerman, and G. Shafer, "Algorithmic learning in a random world," 2005.

[2] Y. Romano, E. Patterson, and R. J. Candès, "Conformalized quantile regression," 2019.

[3] I. Gibbs and E. J. Candès, "Adaptive conformal inference under distribution shift," 2021.

[4] A. N. Angelopoulos and S. Bates, "Conformal prediction: A gentle introduction," 2023.

[5] R. F. Barber, E. J. Candès, A. Ramdas, and R. J. Tibshirani, "Conformal prediction beyond exchangeability," 2023.

[6] Kiyani, Shayan and Pappas, George J. and Roth, Aaron and Hassani, Hamed, "Decision theoretic foundations for conformal prediction: Optimal uncertainty quantification for risk-averse agents," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 30 943–30 965.

[7] Sha, Lui, "Using simplicity to control complexity," 2001, IEEE Software.

[8] Seto, Daisuke and Krogh, Bruce H. and Sha, Lui and Chutinan, Alongkrit, "The simplex architecture for safe online control system upgrades," 1998, Proceedings of ACC.

TABLE XI: Governance and security ablations for the ORIUS runtime-release contract. These are fail-closed ablations, not performance benchmarks: each row removes or corrupts a release-governance precondition and records the expected denial path.

| **Ablation** | **Removed or corrupted component** | **Expected runtime response** | **Evidence surface** | **Interpretation** |
|---|---|---|---|---|
| Missing model hash | Artifact hash manifest absent in strict mode | Refuse load before deserialization | model-hash test; deployment-security validator | Strict release evidence cannot depend on unverifiable model binaries. |
| Bad certificate signature | Certificate payload or signature tampered | Verification fails closed | certificate-full test; schema validator | Certificate-backed release is meaningful only when invalid signatures deny release. |
| Stale device timestamp | Device request timestamp outside allowed skew | Reject telemetry or acknowledgement | IoT HMAC test; deployment-security validator | Device identity is enforced as a runtime precondition. |
| Replayed device nonce | Previously accepted device nonce reused | Reject replayed device request | IoT HMAC test; deployment-security validator | Strict IoT mode includes replay protection. |
| Invalid fallback action | Fallback fails the domain postcondition | Release-contract validation fails | release-contract test and validator | Fallback remains a typed safety obligation, not an unconditional escape. |
| Corrupted release witness | Compact evidence omits canonical fields | Release-contract validator fails | release-witness corruption test and validator | Claim-facing evidence must expose common release fields across promoted domains. |


[9] Desai, Ankush and Ghosal, Akash and Saha, Indranil and Yang, Yuhong and Dutle, Aaron and Ivanov, Radoslav and Lee, Insup and Seshia, Sanjit A. and Tiwari, Ashish, "Soter: A runtime assurance framework for programming safe robotics systems," 2019, Proceedings of IROS.

[10] Bartocci, Ezio and Falcone, Ylies and Francalanza, Adrian and Reger, Giles, "Introduction to runtime verification," 2018, lecture Notes in Computer Science.

[11] M. Leucker and C. Schallhart, "A brief account of runtime verification," 2009.

[12] Kwon, Minjae and Ingebrand, Tyler and Topcu, Ufuk and Feng, Lu, "Runtime safety through adaptive shielding: From hidden parameter inference to provable guarantees," 2025, arXiv:2506.11033.

[13] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," 2019.

[14] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," 2021.

[15] Fisac, Jaime F. and Akametalu, Anayo and Zeilinger, Melanie N. and Kaynama, Sahar and Gillula, Jeremy and Tomlin, Claire J., "A general safety framework for learning-based control in uncertain robotic systems," 2019, IEEE Transactions on Automatic Control.

[16] Ben-Tal, Aharon and El Ghaoui, Laurent and Nemirovski, Arkadi, "Robust optimization," 2009, princeton University Press.

[17] Rajkumar, Ragunathan and Lee, Insup and Sha, Lui and Stankovic, John, "Cyber-physical systems: The next computing revolution," 2010, Proceedings of DAC.

[18] A. Teixeira, I. Shames, H. Sandberg, and K. H. Johansson, "A secure control framework for resource-limited adversaries," 2015.

[19] Basseville, Michele and Nikiforov, Igor V., "Detection of abrupt changes: Theory and application," 1993, prentice Hall.

[20] Pasqualetti, Fabio and Dorfler, Florian and Bullo, Francesco, "Attack detection and identification in cyber-physical systems," 2013, IEEE Transactions on Automatic Control.

[21] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, "Distribution-free predictive inference for regression," 2018.

[22] M. Zaffran, O. Féron, Y. Goude, J. Jolivet, and A. Dieuleveut, "Adaptive conformal predictions for time series," 2022.

[23] Havelund, Klaus and Rosu, Grigore, "An overview of the runtime verification tool java pathexplorer," 2004, formal Methods in System Design.

[24] Page, E. S., "Continuous inspection schemes," 1954, biometrika.

[25] ——, "A test for a change in a parameter occurring at an unknown point," 1955, biometrika.

[26] Hinkley, David V., "Inference about the change-point in a sequence of random variables," 1971, biometrika.

[27] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," 2008.

[28] Breunig, Markus M. and Kriegel, Hans-Peter and Ng, Raymond T. and Sander, Jorg, "Lof: Identifying density-based local outliers," 2000, Proceedings of SIGMOD.

[29] R. Bloem, B. Könighofer, R. Könighofer, and C. Wang, "Shield synthesis: Runtime enforcement for reactive systems," 2015.

[30] Caesar, Holger and Kabzan, Juraj and Tan, Kok Seang and Fong, Whye Kit and Wolff, Eric and Lang, Alex and Fletcher, Luke and Beijbom, Oscar

TABLE XII: Falsification boundaries for the ORIUS claim. These rows define what would break the present bounded-predeployment claim and how the repo currently guards or scopes that risk.

| **Failure mode** | **What would falsify the current claim** | **Current guardrail** | **Boundary or next step** |
|---|---|---|---|
| Coverage miss exceeds $\alpha$ | Empirical or certified coverage miss rate exceeds the theorem/reporting budget on a promoted row | Coverage/calibration artifacts plus the T11 coverage obligation | Recalibrate uncertainty sets or demote the affected row until coverage is restored. |
| Safe-set adapter is wrong | Domain audit shows $C(x)$ excludes a truly safe action or includes an unsafe action | Adapter-validity assumption and domain runtime-contract witness | Correct the adapter and rerun domain-specific runtime evidence. |
| Fallback is not admissible | Fallback violates true-state safety or operational admissibility under a promoted scenario | Release-contract postcondition tests and fallback-validity theorem rows | Replace fallback, deny release, or narrow the claim boundary. |
| Repair leaves the safe core | Repaired action fails membership in the uncertainty-set safe-action intersection | Canonical release evidence and T11 repair-membership obligation | Fail closed and rerun repair validation before promotion. |
| Audit chain is broken | Certificate hash, signature, or append-only chain cannot be verified for a promoted release row | Certificate schema and deployment-security validators | Do not use the row as claim-governing evidence until provenance is repaired. |
| Utility collapses to shutdown | ORIUS matches a fail-safe reference only by always braking, alerting, or shutting down | Utility-preserving safety scorecard | Report the row as safe but vacuous; improve controller/fallback policy before claiming non-vacuity. |
| Closed-loop dynamics diverge | nuPlan, CARLA, or physical loop reveals behavior not represented by bounded replay/surrogate dynamics | Validation-boundary table and external-validation manifest | Promote only after closed-loop evidence exists; otherwise keep the replay boundary explicit. |
| Clinical context differs | Prospective/site clinical use has different workflow, population, or utility costs than retrospective monitoring | Clinical boundary table and source/time-holdout evidence | Require prospective/site validation before clinical-use claims. |


and Omari, Sammy, "nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles," 2021, arXiv:2106.11810.

[31] Karnchanachari, Napat and others, "Towards learning-based planning: The nuplan benchmark for real-world autonomous driving," 2024, arXiv:2403.04133.

[32] Peng, Mingxing and Yao, Ruoyu and Guo, Xusen and Ma, Jun, "nuplan-r: A closed-loop planning benchmark for autonomous driving via reactive multi-agent simulation," 2025, arXiv:2511.10403.

[33] Johnson, Alistair E. W. and Bulgarelli, Lucas and Shen, Lu and Gayles, Anna and Shammout, Ahmad and Horng, Steven and Pollard, Tom and Hao, Shirly and Moody, Benjamin and Gow, Brian and others, "Mimic-iv, a freely accessible electronic health record dataset," 2023, scientific Data.

[34] Johnson, Alistair E. W. and Bulgarelli, Lucas and Pollard, Tom J. and Celi, Leo Anthony and Mark, Roger G., "Mimic-iv version 3.1," 2024, physioNet.

[35] Huang, Yong and Yang, Zhongqi and Rahmani, Amir M., "Mimic-sepsis: A curated benchmark for modeling and learning from sepsis trajectories in the icu," 2025, IEEE BHI / OpenReview.

[36] A. D. Ames, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs with application to adaptive cruise control," 2014.

[37] D. Q. Mayne, M. M. Seron, and S. V. Raković, "Robust model predictive control of constrained linear systems with bounded disturbances," 2005.

[38] W. Langson, I. Chryssochoos, S. V. Rakovčić, and D. Q. Mayne, "Robust model predictive control using tubes," 2004.

[39] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, "Model predictive control: Theory, computation, and design," 2017.

[40] Camacho, Eduardo F. and Bordons, Carlos, "Model predictive control," 2013, springer.